

오늘의 작업기록 — 2026년 7월 7일

ledger-api — POST /transfer: mini-ledger의 Bank가 진짜 API가 된 날 (m105 완주)

(도메인은 한 줄도 안 바꿨다 — 호출자만 Main에서 curl로 갈아끼운 날)

한눈에

항목	내용
목표	m105 조각 2 — Bank를 HTTP에 태워 POST /transfer, 잔액 변경 확인
결과	m105 완주 ✓ — 역등성이 HTTP에서 실전 작동, update.py --done m105
커밋	5d4d4de (server.log 정리) → 이식/record → d0d6f9c "add POST /transfer wired to mini-ledger Bank (verified: idempotent over HTTP)"
환경 명세	Java 21.0.11 · Gradle 9.5.1 · Spring Boot 4.1.0 · Tomcat 11.0.22 (VPS Ubuntu 24.04)
오늘의 그림	5단계 중 ④ 호출 한 칸만 교체 : Main.java → curl→컨트롤러. 도메인(검증·이체·수수료·기록·역등성·복식부기) 수정 0줄
★ 처음 만난 개념	record (DTO 그릇) · @RequestBody+잭슨 (JSON 자동 변환 = 마법 ④) · @PostMapping (상태를 바꾸는 요청) · 상태를 가진 컨트롤러 (요청 간 Bank 유지) · curl POST 플래그 · git rm --cached · sed 'li' · 부모·자식 프로세스
검증 3연타	key-1 이체됨(a 1080) → key-1 재시도 잔액 불변(역등성) → key-2 정상 처리(a 60) + "돈 보존됨" ×2
트러블 3건	IntelliJ 먹통→VPS nano 전환 · 404=파일 부재(nano 저장 실패) · 서버 내림 판정=ss 빈 출력
다음	예외→HTTP 상태코드 번역(잔액 부족이 500으로 새는 것) · 복기 예약(원자성 트래이스+checkBalance Map)

어제 띄운 서버는 인사(/health)만 했다. 오늘은 **진짜 일**을 시켰다 — 6/19부터 쌓은 Bank(원자성 검증·역등성·복식부기까지 든 그 원장)를 그대로 이식해, JSON 요청 한 방으로 이체가 처리되게 했다. 핵심은 **도메인을 한 글자도 안 고쳤다**는 것. Bank를 깔때기로 만들어둔 설계가 오늘 값을 했다 — 호출자만 갈아끼우면 되는 구조였다. 그 리고 6/29에 Set 하나로 만든 역등성이, 오늘 같은 키를 두 번 보내자 잔액이 안 움직이는 것으로 — **Stripe·토스가 쓰는 바로 그 API 패턴**으로 — 실전 검증됐다.

0. 오늘의 그림 — 바뀐 것은 딱 한 칸

이체 한 건이 처리되는 흐름을 다섯 칸으로 보면:

[① 요청 만들기] → [② 값 전달] → [③ 도메인 실행] → [④ 호출자] → [⑤ 결과 확인]

지금까지: ④ = Main.java가 bank.transfer("a","b",1000,"key-1")를 코드로 직접 호출
오늘부터: ④ = curl(JSON 텍스트) → 스프링(변환) → TransferController → bank.transfer(...)

- ③ **도메인 실행부(Bank·Transaction·예외들)**는 **오늘 수정 0줄**. 이식하며 각 파일 첫 줄에 package를 얹은 게 전부다. 검증→실행 순서(원자성), usedKeys(역등성), checkBalance(복식부기)가 전부 그대로 HTTP 아래에서 돈다.
- **Main.java는 안 가져왔다** — Main의 역할(호출·시나리오 구성)을 이제 **curl이 대신**하기 때문이다. mini-ledger 본진은 졸업작으로 보존.
- 이게 되는 이유: 로직을 Bank라는 **깔때기 하나**에 모아뒀기 때문. 입구(호출자)는 Main이든 컨트롤러든 테스트든 갈아끼울 수 있다. 오늘이 그 설계의 첫 실전 증명이다.

재현 타임라인 (오늘 실제 순서 · UTC)

~10:26 세션 1 — Step 0(server.log 정리, 5d4d4de) + Step 1(도메인 6파일 이식, package 일괄, compileJava BUILD SUCCESSFUL in 36s = 끝는 지점 A)
Step 2(TransferRequest record, BUILD 7s)

중간 사지방 PC에서 IntelliJ clone 시도 → Gradle sync 중 IntelliJ 먹통
→ 판단: 오늘 편집은 VPS nano로 (트러블 3-A)

~11:20 세션 2 — TransferController 작성(첫 시도→교정 2곳, PART 2 Step 3)

11:24:53 bootRun 백그라운드 기동, Started in 3.789s (java PID 1178607)

11:28:00 POST /transfer → ★ 404 Not Found (트러블 3-B 시작)
→ ls: TransferController.java 부재 발견(nano 저장 실패) → EOF 재작성
→ grep -c 판정 6 → kill %1 → 재기동

11:32:17 Started in 4.702s (java PID 1178826) — 이번엔 컨트롤러 포함

직후 검증 3연타: key-1 이체 → key-1 재시도(불변) → key-2 이체
grep "돈 보존" ×2 확인
kill %2 → ss로 자식 java 종료 확인(빈 출력) (트러블 3-C)

직후 커밋 d0d6f9c + push → update.py --done m105 → grep "status": "done" 판정

PART 1 — ★ 오늘 처음 만난 개념들 (폴코드 + 정밀 해설)

오늘은 mini-ledger(순수 자바)에서 스프링으로 넘어가는 전환기라 처음 보는 것이 많았다. 이 파트가 오늘 기록의 본체다.

1-1. record — "그릇 클래스"를 한 줄로 만들어주는 자바 문법

작성한 폴코드 (`TransferRequest.java` 전체):

```
package com.ledger.api;

public record TransferRequest(String fromId, String toId, long amount, String idempotencyKey) {}
```

이게 뭐가. `record` 는 "데이터를 담기만 하는 클래스"를 선언 한 줄로 만들어주는 자바 문법이다. 저 한 줄을 컴파일하면 자바가 뒤에서 **자동으로 만들어주는** 것이 넷이다:

1. `private final` 필드 4개 (fromId, toId, amount, idempotencyKey)
2. 그 4개를 전부 받는 생성자
3. 각 필드를 꺼내는 접근자 4개
4. equals / hashCode / toString (덤)

내가 이미 손으로 만들어본 패턴이다. Transaction 클래스를 짤 때 정확히 이 구조를 전부 수동으로 쳤다. record 한 줄이 대체하는 것을 손으로 쓰면 이렇다:

```
// ↓ record 한 줄이 대체하는 "손으로 쓴 버전" (Transaction을 짤 때 내가 쳤던 그 패턴)
public class TransferRequest {
    private final String fromId;
    private final String toId;
    private final long amount;
    private final String idempotencyKey;

    public TransferRequest(String fromId, String toId, long amount, String idempotencyKey) {
        this.fromId = fromId;
        this.toId = toId;
        this.amount = amount;
        this.idempotencyKey = idempotencyKey;
    }

    public String fromId() { return fromId; }
    public String toId() { return toId; }
    public long amount() { return amount; }
    public String idempotencyKey() { return idempotencyKey; }
    // + equals / hashCode / toString
}
```

→ 약 20여 줄이 괄호 안 선언 하나로 접힌다. 즉 **record**는 새 개념이 아니라, 내가 아는 패턴(불변 데이터 그릇)의 자동화다. 낯설게 느껴졌지만 실체는 이미 손에 익은 구조였다.

★ 함정 — 접근자 이름이 게터와 다르다:

```
request.getFromId()    // ✗ record에는 이런 메서드가 없다 (컴파일 에러)
request.fromId()        // ✓ get 없이 "필드명 그대로 + 괄호"
```

게터 관례(getXxx)에 익숙해진 손이 그대로 나가면 바로 에러다. record는 **필드명 자체가 접근자 이름**이다. 컨트롤러 몸통에서 이 접근자로 값을 꺼낸다.

왜 불변인가. record는 만들어진 뒤 값을 못 바꾼다(setter가 없다). 요청 데이터가 처리 도중 바뀌면 안 되니 이게 맞다 — Transaction을 불변으로 설계했던 것과 같은 철학이 여기서도 통한다.

왜 여기 들어갔나 — DTO라는 역할. curl이 보내는 JSON(`{"fromId":"a",...}`)은 그냥 텍스트다. 이걸 자바 세계에서 다루려면 **타입 있는 그릇**이 필요하다. 이런 "요청/응답 데이터 전용 그릇"을 실무에서 **DTO**(Data Transfer Object)라 부른다. 도메인 객체(Bank, Account)와 달리 로직이 없고 **데이터 운반만** 한다 — 그래서 record가 딱 맞는 자리다. 오늘 만든 TransferRequest가 내 첫 DTO다.

1-2. @RequestBody + 잭슨 — JSON 텍스트가 자바 객체가 되는 마법 ④

컨트롤러 시그니처:

```
@PostMapping("/transfer")
public String transfer(@RequestBody TransferRequest request) { ... }
```

이게 뭐가. `@RequestBody` 는 "HTTP 요청의 본문(body)을 이 매개변수의 타입으로 **변환해서 꽂아라**"는 스티커다. 내가 파싱 코드를 한 줄도 안 썼는데, 메서드가 시작될 때 request에는 이미 완성된 TransferRequest 객체가 들어와 있다. 실제로 일어나는 일을 단계로 펼치면:

```
curl이 보낸 것: {"fromId":"a","toId":"b","amount":1000,"idempotencyKey":"key-1"} ← 그냥 텍스트
↓ ① 스프링이 요청 헤더 Content-Type: application/json 을 보고 "본문이 JSON이구나" 판단
↓ ② 잭슨(Jackson) — 스프링에 내장된 JSON 변환기 — 이 텍스트를 파싱
↓ ③ JSON의 키 이름(fromId)과 record의 필드명(fromId)을 짝지음
↓ ④ 사실상 new TransferRequest("a","b",1000,"key-1")를 대신 호출
transfer() 메서드 시작 시점에 request 매개변수에 완성된 객체가 꽂혀 있음
```

★ **핵심 규칙: JSON 키 이름 = record 필드명.** 이 이름 일치가 매핑의 계약이다. 만약 JSON에서 `from_id` 로 보내면 fromId와 짝이 안 맞아 그 자리는 null(참조형)/0(숫자형)으로 조용히 남는다 — 에러도 안 나고 값만 빠지는, 제일 잡기 어려운 유형의 버그다. 그래서 이름을 맞추는 게 규칙이 아니라 계약이다.

왜 마법 목록에 넣나. 스프링이 대신해주는 것 목록(어제 만든 것)에 오늘 하나가 추가된다:

- ① 내장 웹서버(Tomcat) 기동 — 어제
- ② 컴포넌트 스캔(@ 스티커 읽기) — 어제
- ③ URL → 메서드 자동 연결 — 어제
- ④ JSON ↔ 자바 객체 자동 변환 — ★ 오늘 (잭슨)

이 넷이 "스프링이 나 대신 해주는 일"이고, 8월에 이 마법의 아래층(요청 받는 서버)을 직접 만들 때 이 목록이 지도가 된다.

1-3. @PostMapping — GET과 POST는 의미가 다르다

- **GET** (어제 /health): "조회한다". 본문이 없다. `curl URL` 만 치면 기본이 GET이다.
- **POST** (오늘 /transfer): "생성/변경한다". 본문에 데이터를 실어 보낸다.

왜 **transfer는 POST인가**. 이체는 서버의 상태(잔액)를 바꾸는 요청이다. HTTP에서 상태를 바꾸는 요청은 POST(류)로 보내는 게 의미론이다 — "조회는 GET, 변경은 POST"가 첫 원칙. (참고 지형: 수정 전용 PUT, 삭제 전용 DELETE도 있다. 지금은 GET/POST 둘이면 충분하고, 나머지는 만나면 그때.)

`@PostMapping("/transfer")` = "POST /transfer 요청이 오면 이 메서드를 실행하라." 어제 @GetMapping의 POST판이다. 어제 GET /transfer로 쳤다면? — @PostMapping만 있으니 그 경로의 GET은 등록이 없어 405(Method Not Allowed) 계열로 거절된다. **경로뿐 아니라 방식까지 맞아야** 메서드에 도달한다.

1-4. curl POST — 플래그 세 개가 각각 하는 일

```
curl -X POST http://localhost:8080/transfer \
-H "Content-Type: application/json" \
-d '{"fromId":"a","toId":"b","amount":1000,"idempotencyKey":"key-1"}'
```

- **-X POST** — 요청 방식 지정. 안 쓰면 GET이다(어제 /health가 그랬다).
- **-H "Content-Type: application/json"** — "본문이 JSON"이라는 **라벨(헤더)**. 1-2의 ①단계가 정확히 이 헤더를 본다. **이게 없으면** 스프링이 본문을 JSON으로 해석하지 않아 415(Unsupported Media Type) 계열로 거절된다 — 본문이 멀쩡해도 라벨이 없으면 안 읽는다.
- **-d '...'** — **본문(body)** 자체. 여기 실린 텍스트가 잭슨을 거쳐 record가 된다. (JSON 따옴표는 곧은따옴표여야 한다 — 폰 스마트 구두점 주의.)

어제 GET은 플래그 없이 한 줄이었는데 오늘 셋이 붙은 이유 = "방식이 다르고(POST), 본문이 있고(-d), 그 본문의 형식을 알려야 해서(-H)"다.

1-5. ★ 상태를 가진 컨트롤러 — 오늘의 개념 핵심

작성한 풀코드 (`TransferController.java` 전체, 커밋 d0d6f9c):

```
package com.ledger.api;

import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RestController;

@RestController
public class TransferController {

    private final Bank bank = new Bank();

    public TransferController() {
        bank.createAccount("a", "이름", 2100);
        bank.createAccount("b", "이름", 3500);
        bank.createAccount("platform", "플랫폼", 0);
    }

    @PostMapping("/transfer")
    public String transfer(@RequestBody TransferRequest request) {
        bank.transfer(request.fromId(), request.toId(), request.amount(), request.idempotencyKey());
        return "a: " + bank.getAccount("a").getBalance()
            + ", b: " + bank.getAccount("b").getBalance()
            + ", platform: " + bank.getAccount("platform").getBalance();
    }
}
```

줄 단위 해부 — 뭐가 왜 들어갔나:

@RestController — 어제와 동일: 스프링이 부팅 때 컴포넌트 스캔(마법 ②)으로 이 스티커를 발견하고 "HTTP 담당 클래스"로 등록한다. 이 등록 덕분에 아래 @PostMapping이 URL 표에 오른다.

private final Bank bank = new Bank(); — Bank를 **필드**로 둔다. 여기가 오늘의 개념 핵심이다. - 컨트롤러 객체는 서버가 뜰 때 스프링이 **딱 한 번** 만들고, 서버가 사는 동안 유지한다. 따라서 이 bank도 **한 번 만들어져 계속 산다**. - 그 결과: **요청이 100번 와도 전부 같은 bank를 쓴다**. 잔액·history·usedKeys가 **요청과 요청 사이에 이어진다**. 수명을 그림으로 보면:

```

서버 부팅
└─ 스프링 컴포넌트 스캔 → @RestController 발견
    └─ new TransferController() ← 딱 1회
        └─ 생성자 실행: bank 생성 + 계좌 3개 등록 ← 딱 1회

요청 1 (key-1) └─
요청 2 (key-1) └─▶ 셋 다 "같은 bank"를 씀 (필드라서)
요청 3 (key-2) └─

서버 종료(kill) → bank와 모든 상태 소멸

```

Main 시절과의 결정적 차이:

	mini-ledger (java Main)	ledger-api (서버)
Bank 생성	실행할 때마다 새로	부팅 때 딱 한 번
상태 수명	실행 끝나면 소멸	서버가 사는 동안 유지
두 번째 호출	첫 호출을 기억 못 함	첫 호출의 usedKeys가 살아있음

→ 오늘 검증이 이 개념의 증명이다. key-1 2차 요청이 막힐 수 있었던 건, 1차 때 `usedKeys.add("key-1")` 로 저장한 그 Set이 2차 요청 시점에도 **같은 bank 안에 살아있었기** 때문이다. Main 시절이었다면 실행마다 새 Bank라 막을 방법 자체가 없었다. "상태를 가진 서버"가 잔액 불변이라는 숫자로 눈앞에 증명된 것.

한계 — 영속성이 없다 (인지만). 이 상태는 **메모리에만** 있다. 서버를 재시작하면 a의 잔액도, 두 건의 history도, usedKeys도 **전부 증발**한다. "깎다 커도 남는 기록"을 영속성(persistence)이라 하고, 지금은 그게 없다. 푸는 도구가 나중의 DB/JPA이며, **8월 Layer 0의 WAL이 정확히 이 문제**(디스크에 먼저 기록해 죽어도 남기기)를 다룬다. 오늘 "재 시작하면 증발"을 직접 인지해둔 게 그때의 동기가 된다.

생성자의 createAccount 3개 — 서버 부팅 시 1회 초기화다. - a는 이체 1000 + 수수료 20 = 1020을 감당해야 하니 2100 (7/5 원자성 검증 `getBalance() < amount + fee` 를 통과하도록). - **platform 포함이 함정 회피 포인트** — Bank.transfer가 내부에서 `getAccount("platform")` 을 부른다(수수료 입금). 빼먹으면 첫 이체가 AccountNotFoundException으로 터진다. 7/5에 Main에서 만난 그 함정을, 오늘은 기억해서 처음부터 넣었다.

메서드 몸통 2줄: - ① `bank.transfer(request.fromId(), ...)` — request에서 record 접근자로 4개 값을 꺼내 도메인에 전달. Main이 `"a", "b", 1000, "key-1"` 을 직접 쓰던 것과 유일한 차이가 이것 — **값이 이제 request 안에 있으니 꺼내서 넘긴다**. - ② `return "a: " + ... getBalance() ...` — 세 계좌 잔액을 문자열로 조합해 반환. **return 값이 곧 HTTP 응답 본문**(어제 배운 것)이므로, m105 완료 조건("잔액 변경 확인")이 응답에 그대로 보이게 설계했다. mini-ledger Main에서 println으로 찍던 그 조합을 return으로 옮긴 것.

1-6. git rm --cached — 이미 커밋된 파일을 gitignore로 빼는 법

상황. 어제 server.log가 커밋에 딸려 들어갔다. .gitignore에 `server.log` 를 추가했는데 — 그것만으론 안 빠진다.

메커니즘. .gitignore는 "아직 추적되지 않은 파일을 새로 추적하지 말라"는 규칙이다. server.log는 이미 커밋돼 **추적 목록(인덱스)에 올라가** 있으므로 ignore가 무시된다. 그래서:

```
git rm --cached server.log
```

- `--cached` = "**추적 목록에서만** 제거하라" — 디스크의 실제 파일은 그대로 남는다(서버 로그는 계속 쌓여야 하니까).
- 추적에서 빠진 순간부터 .gitignore가 먹기 시작한다. → "이미 커밋된 파일을 ignore로 빼는" 표준 2단계: **ignore에 추가 + rm --cached**. (그냥 `git rm` 이면 디스크 파일까지 지워진다 — cached가 핵심이다.)

1-7. sed '1i' — 파일 첫 줄 앞에 삽입 (package 일괄 작업)

```
sed -i '1i package com.ledger.api;' Account.java
```

- `-i` = 파일을 **제자리에서** 수정
- `1i` = **1번 줄 앞에**(i = insert) 뒤의 텍스트를 삽입
- package 선언은 파일 **최상단**이어야 하므로 '1'가 정확한 자리다.

여섯 파일에 for 루프로 일괄 적용한 것은, 어제 배운 규칙("package = 폴더 경로와 일치")의 자동화다. 손으로 여섯 번 열 일을 한 줄 루프로.

1-8. 부모·자식 프로세스 — gradle이 java를 "자식으로" 띄운다

`./gradlew bootRun &` 로 서버를 띄우면 프로세스가 **둘** 생긴다:

```
[백그라운드 작업 %N] gradle(래퍼)  ← 부모
└─ java(스프링 서버, 실제 8080 점유) ← 자식
```

[2] 1178776 (gradle)과 로그의 **PID 1178826** (java)이 달랐던 이유가 이것이다. `kill %2` 는 부모(작업)를 죽이는데, 자식 java는 **시차를 두고** 정리되거나 드물게 남을 수 있다. → 그래서 서버를 내렸는지의 **판정 기준은 kill 명령이 아니라 ss의 빈 출력**이다(트러블 3-C에서 실제로 겪음).

PART 2 — 작업 흐름 (유의미한 명령 + 판정, 첫 시도→교정 포함)

Step 0 — server.log 추적 제거 (어제 숙제)

```
cd ~/ledger-api && echo "server.log" >> .gitignore
git rm --cached server.log && git commit -am "ignore server.log"
```

판정:

```
rm 'server.log'
[main 5d4d4de] ignore server.log
 2 files changed, 1 insertion(+), 27 deletions(-)
```

(원리는 1-6. 이후 서버 로그가 아무리 쌓여도 커밋에 안 딸려온다.)

Step 1 — 도메인 이식: 여섯 파일 + package 일괄

```
cd ~/mini-ledger && git pull
cp Account.java Bank.java Transaction.java TransactionType.java \
  AccountNotFoundException.java InsufficientBalanceException.java \
  ~/ledger-api/src/main/java/com/ledger/api/

cd ~/ledger-api/src/main/java/com/ledger/api
for f in Account Bank Transaction TransactionType AccountNotFoundException InsufficientBalanceException; do
  sed -i '1i package com.ledger.api;' "$f.java"
done
head -2 *.java
cd ~/ledger-api && ./gradlew compileJava
```

판정:

```
==> Account.java <==           (여섯 파일 전부 동일 패턴)
package com.ledger.api;
public class Account {
...
BUILD SUCCESSFUL in 36s        ← 이식 성공 = 끊는 지점 A
```

- Main.java 제외 이유는 0장. 원리(sed '1i')는 1-7.
- **BUILD SUCCESSFUL의 의미:** mini-ledger의 도메인 전체 — 원자성 검증이 든 transfer, refund, checkBalance까지 — 가 스프링 프로젝트 안에서 컴파일된다는 것. 이 시점에 커밋하고 끊어도 완결이었다(오늘은 계속 감).

Step 2 — TransferRequest record 작성

폴코드와 해설은 **1-1**. 작성 후:

```
./gradlew compileJava # 판정: BUILD SUCCESSFUL in 7s
```

Step 3 — TransferController: 첫 시도 → 교정 2곳 → 완성 (솔로)

뼈대(어디를 채울지 표시된 골격)를 받고 조립은 직접. 첫 시도 원문:

```
@PostMapping("/transfer")
public String transfer(@RequestBody TransferRequest request) {
    bank.transfer
    bank.getAccount("a").getBalance();
    return " ";
}
```

첫 시도에서 이미 맞았던 것 (7개): 임포트 3줄 · @RestController · 클래스 골격 · `private final Bank bank = new Bank();` · @PostMapping 시그니처 · @RequestBody 매개변수 · **계좌 3개 초기화(platform 포함 — 7/5 합정을 기억해서 회피)**. 즉 스프링 쪽 새 문법은 전부 첫 시도에 통과했다.

교정 ① — `bank.transfer` 에 인자가 없었다. transfer는 4개 인자가 필요하다(mini-ledger에서 `bank.transfer("a","b",1000,"key-1")` 로 불렀던 그 시그니처). 지금은 그 값들이 **request 안에** 있으므로 record 접근자로 꺼내 넘긴다:

```
bank.transfer(request.fromId(), request.toId(), request.amount(), request.idempotencyKey());
```

막힌 지점의 본질: "값을 직접 쓰던 것 → request에서 꺼낸다"는 한 뼔 차이. 그걸 넘으니 나머지는 아는 자바였다.

교정 ② — `return`이 비어 있었다 (`return " ";`). 그리고 `bank.getAccount("a").getBalance();` 한 줄은 결과를 아무 데도 안 써서 무의미했다(호출만 하고 버림). `getBalance`의 결과를 **return 문자열에 담아야** 응답에 잔액이 실린다:

```
return "a: " + bank.getAccount("a").getBalance()
+ ", b: " + bank.getAccount("b").getBalance()
+ ", platform: " + bank.getAccount("platform").getBalance();
```

(println으로 찍던 조합을 return으로 — 서버에선 return이 곧 응답이므로.)

완성본 폴코드와 줄 단위 해설은 **1-5**.

```
./gradlew compileJava # 판정: BUILD SUCCESSFUL in 8s
```

Step 4 — 기동 → 검증(PART 4) → 종료(트러블 3-C)

Step 5 — 검증된 것만 커밋 + 마일스톤 done

```
git add . && git commit -m "add POST /transfer wired to mini-ledger Bank (verified: idempotent over HTTP)" && git push
```

판정:

```
[main d0d6f9c] add POST /transfer wired to mini-ledger Bank (verified: idempotent over HTTP)
1 file changed, 25 insertions(+)
create mode 100644 src/main/java/com/ledger/api/TransferController.java
cc07b06..d0d6f9c main -> main
```

```
cd ~/backend-journey && python3 update.py --done m105
grep -A6 '"id": "m105"' roadmap.json | grep status
```

판정:

```
✓ done 처리: m105
→ 푸시 완료 · https://taewoo-won.github.io/backend-journey/ (1분 뒤 반영)
"status": "done"
```

★ **m105 done**은 sed가 아니라 **update.py --done**으로. roadmap.json은 cron의 update.py가 1분마다 **재생성**하는 파일이라, 손 sed는 다음 틱에 덮인다(7/6에 m103으로 실제 당한 것). **자동 writer가 있는 파일은 그 인터페이스로만 쓴다** — 그리고 실행 후 grep으로 done을 **눈으로 판정**하는 것까지가 절차다(상태 변경은 뭐든 확인하고 넘어감

다).

PART 3 — 트러블슈팅 3건 (증상 → 추적 → 원인 → 해결 → 교훈)

3-A. IntelliJ 먹통 → 오늘 편집은 VPS nano로 (우회 판단)

증상. 사지방 PC에서 ledger-api를 처음 열려고 clone(File → New → Project from Version Control, URL 입력, Shallow clone은 **체크 해제** — 작은 repo라 아낄 게 없고 히스토리를 읽는다). Clone 직후 Gradle sync가 돌며 **IntelliJ만 완전 먹통** — 커서도 크롬도 정상인데 IDEA 창만 클릭 불가.

원인. IntelliJ + Gradle sync(의존성 다운로드 + 전체 인덱싱)가 이 PC 사양에 과부하. mini-ledger(순수 자바, 파일 몇 개)는 가벼워 괜찮았지만 **스프링 프로젝트는 무겁다** — 어제 bootRun 16분(Gradle 최초 다운로드)과 같은 뿌리다. "컴이 약하다" = 설치 문제가 아니라 **성능 문제**(새로 깔 것은 없다, 이미 다 있다).

판단 (A/B 갈림). - A: IntelliJ 씨름 계속 — sync 끝나길 기다리거나 강제 종료 후 재시도. 또 먹통 위험. - B: **오늘 파일은 VPS nano로 편집** — TransferController는 짧고, 어차피 실행(bootRun/curl)이 VPS라 **편집·실행을 한곳에서** 하는 게 마찰이 적다. → **B 선택.** 단 이것은 **일회성 우회**다 — IntelliJ를 버리는 게 아니고(코드는 전부 GitHub/VPS에 있어 읽을 것도 없다), PC 성능 튜닝은 **별도 과제**로 미뤄 오늘 진도와 섞지 않는다.

교훈. 실행 환경이 VPS인 프로젝트는 편집도 VPS가 마찰이 적다. 그리고 도구와 씨름하다 진도를 잃지 말 것 — 우회하고, 도구 문제는 따로 잡는다.

3-B. 404 Not Found — 진짜 원인은 "파일이 없었다"

증상. 서버 정상 기동(**Started ... in 3.789 seconds**) 후 첫 POST:

```
{"timestamp":"...", "status":404, "error":"Not Found", "path":"/transfer"}
```

추적. 서버는 떴고 compileJava도 SUCCESSFUL이었는데 404 = **"/transfer라는 경로가 서버에 등록돼 있지 않다**"는 뜻. 등록이 안 됐다면 컨트롤러가 스캔에서 빠졌다는 것 → 폴더를 직접 확인:

```
ls ~/ledger-api/src/main/java/com/ledger/api/
# → TransferRequest.java는 있는데 TransferController.java가 목록에 없다!
find ~/ledger-api -name "*ransfer*"
# → TransferRequest.java 뿐 — Controller 부재 확정
```

원인. nano에서 **짤 파일이 저장되지 않았다**(Ctrl+O 누락 추정). 파일 자체가 없으니 스캔할 대상도, /transfer라는 URL도 존재하지 않았다.

여기서 배운 것 둘: 1. **404의 정확한 뜻** = "그 경로가 등록돼 있지 않다." 서버 다운(연결 거부)도, 코드 버그(500)도 아니고 — **경로의 부재**. 2. **"BUILD SUCCESSFUL"**은 '있는 파일들'의 성공이지 '있어야 할 파일'의 존재 증명이 아니다. 빌드 성공을 보고 파일이 있다고 **가정**했다 — 판정 순서는 파일 존재(ls/grep)부터. 이틀째 같은 문장이다: 가정 말고 확인.

해결. EOF heredoc으로 재작성(EOF는 저장 단계 자체가 없어 이 실수가 원천 차단된다) → 판정 **grep -c "PostMapping\|transfer\|RestController" = 6** → **kill %1** 후 재기동 → **Started ... in 4.702 seconds** → 이후 200.

3-C. 서버 내림의 판정은 kill이 아니라 "ss 빈 출력"

증상. 검증 후 **kill %2** 직후 확인:

```
ss -tlnp | grep 8080
LISTEN ... users:((("java",pid=1178826,...))      ← java가 여전히 LISTEN?!
[2]+  Terminated ... ./gradlew bootRun ...      ← gradle은 죽었는데
```

전개. 1-8의 구조 그대로 — kill %2는 **부모(gradle)**를 잡았고, **자식(java)**은 그 순간 아직 살아 보였다. **kill 1178826** 을 치자 **No such process** — 이미 종료돼 있었다(부모 종료가 **시차**를 두고 자식까지 정리한 것; 첫 ss는 그 시차의 잔상). 재확인:

```
ss -tlnp | grep 8080      # → (빈 출력) = 포트 닫힘 확정
```

교훈. 공개 IP(104.236.232.210) 위 서버의 "내렸다" 판정 기준은 **kill 명령의 실행이 아니라 ss의 빈 출력**이다. 부모·자식 구조 때문에 kill 한 방이 즉시 안 먹을 수 있으니, 종료할 때마다 ss로 확인하는 것을 고정 절차로 삼는다.

PART 4 — 검증: m105 완료 조건 (먹등성이 HTTP에서 사는 순간)

```
./gradlew bootRun > server.log 2>&1 &
grep "Started LedgerApi" server.log      # 판정: Started ... in 4.702 seconds
```

3연타 curl과 잔액 변화:

#	보낸 것	응답 (= return 문자열)	의미
1	key-1, a→b 1000	a: 1080, b: 4500, platform: 20	이체 1000 + 수수료 20이 HTTP로 처리됨
2	key-1 그대로 재전송	a: 1080, b: 4500, platform: 20 (불변)	★ 먹등성이 HTTP에서 작동 — usedKeys가 막음
3	key-2, a→b 1000	a: 60, b: 5500, platform: 40	새 키는 정상 처리 — 같은 키"만" 막는다는 증거

각 줄이 증명하는 것: - 1번 — JSON→record→Bank.transfer→잔액 return, 전 파이프라인이 산다. a가 1020 냄(이체+수수료), platform이 20 받음: 7/4~5의 정산·원자성 로직이 HTTP 아래서 그대로. - 2번(하이라이트) — 잔액이 한 푼도 안 움직였다. 1차 때 저장된 usedKeys("key-1")가 같은 bank 안에 살아있어서(1-5의 상태 유지) 2차를 초입에서 걸렀다. Main 시절엔 구조적으로 불가능했던 일이다. - 3번 — 멍등성이 "중복만" 막고 "새 거래"는 통과시킨다는 반대편 증명. a: 1080→60 (또 1020).

돈 보존 산수: a 총 -2040 (1020×2) = b +2000 + platform +40. 두 이체 모두 온전 — 시스템 전체에서 돈이 새지도 생기지도 않았다.

복식부기가 HTTP 뒤에서도 도는가:

```
grep "돈 보존" ~/ledger-api/server.log
# 판정:
돈 보존됨 (합 = 0)
돈 보존됨 (합 = 0)           ← 정확히 두 줄
```

★ "2"라는 숫자 자체가 구조 증명이다. Bank.transfer의 첫 줄이 이거였다:

```
if (usedKeys.contains(idempotencyKey)) { return; }    // 멍등성 관문 — 초입
...
checkBalance();                                     // 복식부기 — 맨 끝
```

→ 막힌 2번 요청은 첫 줄에서 return돼 checkBalance까지 아예 가지 않는다. 실제 이체된 1·3번만 끝까지 가서 "돈 보존됨"을 찍었다. 로그 줄 수 = 실제 실행된 이체 수. 어제까지 만든 원자성·복식부기가 HTTP 층 아래에서 그대로, 정확한 횟수만큼 살아 돈다.

이게 왜 큰가. idempotencyKey 로 중복 요청을 흡수하는 이 구조는 Stripe·토스 결제 API의 실제 계약 그대로다. 네트워크에서 요청은 중복 전송될 수 있다(사용자 재시도, 타임아웃 후 자동 재전송) — 그때 이체가 두 번 되면 사고다. 6/29에 Set 하나로 개념으로 만든 것이, 오늘 "같은 키 두 번 = 잔액 불변"이라는 실전 검증을 통과해 진짜 API 계약이 됐다.

(보너스 관찰 예약) 지금 a=60이라 다음 1000 이체는 잔액 부족이다 — 시도하면 InsufficientBalanceException이 HTTP에서 500(서버 내부 오류)으로 샌다. 클라이언트 잘못(잔액 부족)인데 서버 탓(500)으로 표기되는 것 — "자바 예외를 HTTP 상태코드로 어떻게 번역하나"가 다음 조각이다. 오늘은 위치만 찍어둔다.

5. 재현 절차 요약 (6개월 뒤 따라치기용 — 명령만)

```
# 0) 이미 커밋된 로그 파일을 추적에서 제거
cd ~/ledger-api && echo "server.log" >> .gitignore
git rm --cached server.log && git commit -am "ignore server.log"

# 1) 도메인 이식 + package 일괄 + 컴파일 판정
cd ~/mini-ledger && git pull
cp Account.java Bank.java Transaction.java TransactionType.java \
  AccountNotFoundException.java InsufficientBalanceException.java \
  ~/ledger-api/src/main/java/com/ledger/api/
cd ~/ledger-api/src/main/java/com/ledger/api
for f in Account Bank Transaction TransactionType AccountNotFoundException InsufficientBalanceException; do
  sed -i 'li package com.ledger.api;' "$f.java"; done
head -2 *.java && cd ~/ledger-api && ./gradlew compileJava          # BUILD SUCCESSFUL

# 2) TransferRequest.java / TransferController.java 작성 (PART 1 풀코드)
#   ★ 작성 직후 파일 존재+내용 판정부터:
ls src/main/java/com/ledger/api/ && grep -c "PostMapping" src/main/java/com/ledger/api/TransferController.java

# 3) 기동 + 준비 판정
./gradlew bootRun > server.log 2>&1 &
grep "Started LedgerApi" server.log

# 4) 검증 3연타 (같은 명령을 key만 바꿔서)
curl -X POST http://localhost:8080/transfer -H "Content-Type: application/json" \
  -d '{"fromId":"a","toId":"b","amount":1000,"idempotencyKey":"key-1"}'
# → 한 번 더 그대로(불변 확인) → key-2로(변경 확인)
grep "돈 보존" server.log                                     # 실제 이체 수만큼

# 5) 종료 판정 + 커밋 + 마일스톤
kill %1; ss -tlnp | grep 8080                                # 빈 출력이어야 종료
git add . && git commit -m "add POST /transfer ... (verified)" && git push
cd ~/backend-journey && python3 update.py --done m105
grep -A6 '"id": "m105"' roadmap.json | grep status          # "done" 판정
```

6. 개념 사전 (복기용 압축판 — 정밀 해설은 PART 1)

- record — 불변 데이터 그릇을 한 줄로. 필드·생성자·접근자 자동 생성. 접근자는 fromId() (get 없음).

- **DTO** — 요청/응답 데이터 운반 전용 그릇. 로직 없음. record가 적임.
- **@RequestBody** — 요청 본문(JSON)을 매개변수 타입으로 자동 변환해 주입. 객슨이 수행. 매핑 계약 = JSON 키명 ↔ 필드명 일치.
- **@PostMapping** — POST 요청을 메서드에 연결. 조회=GET(본문 X), 변경=POST(본문 O). 방식까지 맞아야 도달.
- **curl POST** — `-X POST` (방식) / `-H Content-Type` (본문 형식 라벨, 없으면 415) / `-d` (본문).
- **상태를 가진 컨트롤러** — 스프링이 컨트롤러를 부팅 시 1회 생성 → 필드 bank가 요청 간 유지. 한계: 메모리뿐(재시작=증발, 영속성 부재 → DB/JPA · 8월 WAL).
- **git rm --cached** — 추적 목록에서만 제거(디스크 보존). 이미 커밋된 파일에 ignore를 먹이는 표준.
- **sed '1i텍스트'** — 1행 앞 삽입. package 일괄에 사용.
- **부모·자식 프로세스** — gradle(부모)이 java(자식)를 띄움. 종료 판정은 ss 빈 출력.
- **404** — "그 경로가 등록돼 있지 않다." 파일/스캔/매핑 부재를 의심.

7. 산출물

- **커밋:** `5d4d4de` (ignore server.log) → 이식/record → `d0d6f9c` (POST /transfer, verified: idempotent over HTTP) — repo `Taewoo-Won/ledger-api`
- **신규 파일:** `TransferRequest.java`, `TransferController.java` (폴코드 · 해설 PART 1)
- **마일스톤:** **m105 done** — `update.py --done + grep "status": "done"` 판정 완료
- **ledger-api 현재 모습:** GET /health + POST /transfer, 그 아래 mini-ledger 도메인 전체(원자성 · 멍등성 · 복식부기 포함)가 그대로 가동

8. 로드맵 위치와 다음

- **m105 완주.** p1(Foundations)에서 m103(mini-ledger 학습판) · m105(첫 Spring API) 완료 — 7월 관문(m107)은 사실상 조기 달성 상태, 정식 선언만 남음.
- **다음 후보:** ① **예외→HTTP 번역** — 잔액 부족(클라이언트 사정)이 500(서버 탓)으로 새는 것을 400대로 바로잡기 (오늘 위치 찍어둠) ② **복기 예약** — 원자성 트래이스 + checkBalance Map 빈화면 재현 (7/8~9, "Map이 아직 손에 안 붙었다") ③ m106 CodeCrafters Redis (7월 후반 슬롯)
- **8월 예고:** 오늘 "재시작하면 잔액 증발"(영속성 부재)을 직접 인지했다. Layer 0의 WAL이 정확히 이 문제 — 디스크에 먼저 기록해 죽어도 남기기 — 를 다룬다. 오늘의 인지가 그때의 동기다.

9. 검증 질문 (다음 복기용 — 답 적지 말 것)

1. record 한 줄이 자동 생성해주는 것 넷은? 그 한 줄이 대체하는 "손으로 쓴 버전"의 구조를 빈 화면에 재현할 수 있나?
 2. record 접근자 이름 규칙은 게터와 뭐가 다른가? `request.getId()` 를 치면 무슨 일이 나나?
 3. @RequestBody가 하는 일을 4단계로 설명하라. 매핑의 계약(무엇과 무엇의 일치)은? 이름이 어긋난 키는 어떻게 되나?
 4. curl의 `-X`, `-H`, `-d` 는 각각 뭘 지정하나? `-H` 가 없으면 왜/어떻게 거절되나?
 5. 조회와 변경에 각각 어떤 HTTP 방식을 쓰나? transfer가 POST인 이유는? @PostMapping만 있는 경로에 GET을 치면?
 6. Bank를 필드로 둔 것의 의미를 "key-1 2차 요청이 막힌 메커니즘"과 연결해 설명하라. Main 시절엔 왜 구조적으로 불가능했나?
 7. 이 상태의 한계는? "영속성"이 뭐고, 무엇이(8월) 그 문제를 다루나?
 8. "돈 보존됨"이 정확히 2번인 것이 증명하는 구조는? (Bank.transfer의 어느 두 줄과 연결되나)
 9. 오늘 404의 진짜 원인은? "BUILD SUCCESSFUL"이 보증하지 않는 것과, 올바른 판정 순서는?
 0. 공개 IP 서버 "내렸다"의 판정 기준은? kill %N 한 방이 안 먹을 수 있는 구조적 이유는?
1. .gitignore만으로 server.log가 안 빠졌던 이유와, --cached의 정확한 의미는?
 2. m105 done을 sed가 아니라 update.py --done으로 한 이유는? (7/6의 어떤 사고와 연결되나)

10. 회고

오늘 가장 크게 남은 건 두 장면이다.

첫째, **도메인을 한 글자도 안 고치고 API가 됐다.** 프로 여섯 파일을 옮기고 package 한 줄씩 없은 게 전부다. 6/19부터 Bank에 쌓은 검증 · 멍등성 · 복식부기가, 호출자를 Main 에서 curl로 바꾸는 것만으로 HTTP 세계에서 그대로 돌아왔다. "로직을 Bank라는 껍데기에 모아두면 입구는 갈라끼울 수 있다"는 걸 말로만 알다가 오늘 몸으로 확인했다. 컨트롤러를 짜다 막힌 지점도 사실 스프링이 아니었다 — "값을 직접 쓰던 걸 request에서 꺼낸다"는 한 뼘 차이였고, 그걸 넘으니 나머지는 아는 자바였다. 짜기 시작할 때 "하나도 모르겠다"고 느꼈지만 뚜껑을 열어보니 골격 7개를 첫 시도에 다 맞혔더라. **낮섬과 모름은 다르다** — record도 그랬다. 처음 보는 문법인 줄 알았는데, Transaction을 30줄로 손수 썼던 바로 그 패턴의 자동화였다. 전환기의 공포는 대부분 이름이 바뀐 것에 대한 공포지, 내용이 바뀐 것에 대한 공포가 아니었다.

둘째, **key-1을 두 번째 보냈을 때 잔액이 안 움직인 순간.** 그 한 줄이 오늘의 하이라이트다. 서버 안의 Bank가 요청 사이에 살아있어서, 1차 때 저장한 usedKeys가 2차를 초입에서 막았다 — "상태를 가진 서버"라는 개념이 숫자 불변으로 눈앞에 증명됐다. Main 시절엔 실행마다 새 Bank라 이게 구조적으로 불가능했다는 걸 생각하면, 오늘 서버로 넘어온 의미가 정확히 이 한 줄에 있다. 그리고 이게 Stripe · 토스가 쓰는 바로 그 패턴이라는 것 — 6/29에 Set 하나로 만든 멍등성이 오늘 실전 API 계약이 됐다. "돈 보존됨"이 정확히 2 번 찍힌 것까지 — 막힌 요청은 검증까지 안 간다는 구조가 로그 줄 수로 드러난 게 묘하게 통쾌했다.

삼질도 셋 있었고 셋 다 판정 습관이 잡았다. IntelliJ가 멍통일 때 씨름 대신 nano로 우회한 것(진도가 우선, 도구 문제는 별도 과제 — 컴을 버린 게 아니라 오늘만 돌아간 것), 404에서 "빌드 성공 = 파일 존재"라는 가정이 깨진 것(ls부터가 순서였다), kill 후 ss로 재확인해 부모 · 자식 프로세스의 시차를 본 것. 이를 연속 같은 문장이 값을 한다 — **가정 말고 확인.**

한계도 눈에 넣어뒀다. 지금 잔액은 메모리에만 있다. 서버를 껐다 켜면 a의 60원도, 두 건의 기록도, usedKeys도 전부 증발한다. 오늘은 그게 꽤 많은 학습 단계지만, 진짜 원장은 껐다 켜도 남아야 한다 — 8월 WAL이 정확히 그 문제고, 오늘 이 "증발"을 직접 인지한 게 그때의 동기가 될 것이다.

한 줄: 도메인 한 줄 안 바꾸고 호출자만 갈라끼워 Bank를 HTTP API로 — 같은 키 두 번에 잔액이 안 움직이는 순간, 6/29의 멍등성이 Stripe · 토스의 실전 패턴이 됐다. m105 완주.

